DRYDOCK WHITE PAPER SERIES

SDD: Optimal Path to Managing Changed Specifications

Why a changed specification must not rebuild the product, and what a source-driven refit does instead.

Ed Barlow Web Cloud Studio August 2026

Abstract

In specification-driven development the specification is the source of truth and the product is generated from it.

This paper investigates initial build modifications. These tend to result in a full rebuild of the application from specifications and our goal is to manage costs and context for new projects.

New build are never quite correct. There are numerous minor changes the developer will want at the beginning of the project that do not require a complete change ticket cycle.

This paper solves the problem precisely and documents the solution Drydock implements.

Keywords: specification-driven development, source-driven refit, immutable blueprints, change tickets, lineage, dependency graph

The Problem

Drydock imports specifications into a workspace. `drydock analyze` decomposes to stories and `drydock plan` builds blueprints and a graph database around those stories. Each blueprint is a story (or feature) with dependencies and acceptance criteria and other information needed to build.

The user should not edit files which are generated or can be regenerated programatically. The user will probably feel most comfortable with their initial specification. After an initial build, the original specifications authored by the user should be rebuilt with any textual changes the user has made.

A solution needs to handle the following

1) Edits to The Users Specifications 2) Updates of those specifications to our buildable blueprints and graph database 3) Rebuilding from that update

What Does Not Work

Rebuild everything. This is rejected on cost. A full rebuild also discards working, reviewed, scored code and reproduces it in a non deterministic manner requiring a full retest.

Edit the Blueprint in place. This makes the Blueprint, which is a build artifact, the source of truth and the authored specifications a stale copy. It also destroys the property that matters most: that the product can be regenerated from the authoritative source at any time.

Mine the change out of the session. Reading the LLM transcript for what changed produces slop. A working session is exploration, dead ends, and thinking. See *Managing Changes in Specification- Driven Development* [1] for that experiment and its failure.

The Solution — Refit

Define Source of Truth - In the drydock system this is the specifications Keep the specification the user wrote. Freeze the blueprints the build made from it. When the specification changes, write down the change and build only the change.

Four rules do the whole job.

1. The user edits their own file. Nothing else is editable. The build copies the specification when it imports it, and works from the copy. The original stays in the user's hands.

2. Nothing moves until the user says so. Editing the specification does not start a build. The user re-imports when the edit is ready. That is the save button, and it is the line between drafting and committing.

3. Blueprints are frozen. A blueprint is build output. It is never edited, not by hand and not by a model. The only way to change one is to plan it again from the specification.

4. The change becomes a numbered ticket. Re-import compares the new specification to the old copy, finds what moved, and writes one ticket per affected blueprint. The ticket says what the change is and what must now be true. The build reads the blueprint plus its tickets, in number order, and that is the current specification.

Rule 4 is the whole trick. Nothing gets rewritten. Things get appended.

You edit the spec. Import copies it. Refit writes the ticket. Build applies it.

Ordering the Changes

The blueprints were already built, reviewed, and tested. That work stands. Only the tickets get built.

Each blueprint keeps its own numbered list of tickets. Ticket 1 runs after the blueprint. Ticket 2 runs after ticket 1. A ticket may contradict the blueprint or an earlier ticket, and the later one wins.

That is how the user changes their mind in week three without editing anything from week one.

The blueprint plus its tickets, in order, is the current specification.

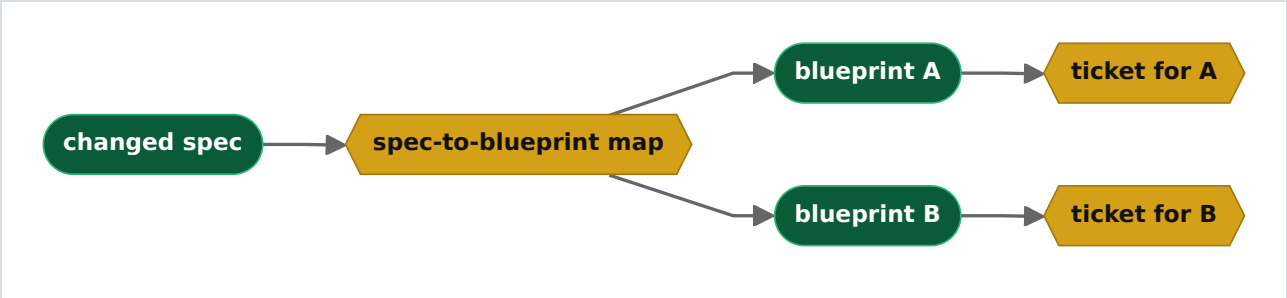
A ticket is just another story in the graph. It has a state, dependencies, and acceptance criteria, so it builds, reviews, and scores like every other story. Nothing new had to be written to run it.

Finding the Right Blueprint

A change to one paragraph of the specification does not touch every blueprint. The build has to know which ones it does touch, or it is back to rebuilding everything.

So the link is written down when the blueprints are planned. Planning already knows it — it read the specification and decided what each blueprint covers. Recording that link costs nothing at plan time and turns "what did I just break" into a lookup.

One paragraph can feed two blueprints. When it does, the change is split, and each ticket belongs to exactly one blueprint. A ticket never spans two.



The map is written at plan time, not worked out at change time.

Who Writes What

The model writes prose. The code writes structure. Mixing the two is where these systems fail.

Job	Done by
Say what changed and what must now be true	Model

Job	Done by
Pick the ticket number and filename	Code
Attach the ticket to the right blueprint	Code
Record what the specification looked like at the time	Code
Check the ticket names a blueprint the map allows	Code
Decide to apply it	The user, by running the build

A model that picks its own numbers will collide. A model that draws its own dependencies will draw the wrong ones. Give it the one job it is good at — writing the change down clearly — and check its answer against the map.

The Rules

All or nothing. A refit either finishes or leaves nothing behind. Half a set of tickets is a broken build. On failure nothing is written and nothing is marked as seen, so running it again does the whole job.

Foundation changes are not refits. Architecture, the data model, and platform rules are context for every blueprint. A change there really does invalidate the build, and no ticket can express it. That case stops, names the file, and asks for a replan.

A deletion is a question. A section that disappears might mean the feature is gone, or it might mean the user sent part of the file. The build asks and records the answer. It never deletes on its own.

Old tickets still run. A later ticket can make an earlier one pointless. There is no reliable way to detect that, so the earlier one is applied anyway. Wasted work is cheaper than a wrong skip.

Starting Over

Tickets are never merged back into the blueprint. Merging is harder than planning and less reliable.

When the chain gets long, re-import the specification and plan it again. The result is clean blueprints and no tickets. It is the same command that made the blueprints the first time, so the reset is always available and always cheap.

That is why nothing warns about ticket count and nothing nags the user to clean up. One rule keeps the reset safe:

> *Every decision goes back into the user's specification. Nothing lives only in a blueprint or a ticket.*

Break that rule and replanning throws away work.

Result

	Before	After
A one-line specification change	Full rebuild	One ticket
Blueprints	Rewritten on every change	Written once
Finding the impact	Read everything and guess	Look it up
Foundation change	Looks like a typo	Stops and asks for a replan
Cleaning up	Merge tickets back	Re-import and plan again

Best practice: freeze the build artifacts, keep the user in their own specification, and turn every edit into a numbered ticket attached to the blueprint it changes. Build the tickets in order.

References

[1] E. Barlow. *Managing Changes in Specification-Driven Development.* Web Cloud Studio, 2026.

[2] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery.* Web Cloud Studio, 2026. https://github.com/webcloudstudio/Drydock